Alogirthm

an Algorithm is a finite set of instruction, following which it is possible to solve a problem

* An infinite loop is not an algorithm as algorithm must have finite termination time

Criterion to be an algorithm →

- inputs
- outputs
- Definitness
- finitness
- effectiveness

Definitness → algorithm should be clear and unambigous like,

divided by zero

add 6 or 7 to x These are unambigious

finitness →

algorithm should halt in a finite state of time

theoritically should not be bound by a time but executes in a finite time

Os is not an algorithm but has amalgamation of algorithm

Areas of study ?

- devising an algorithm [through the requirement of problem statement]
- validating an algorithm [theoritically checking and mathmatically analysing]
- analysing an algorithm → checking memory and time complexity
- testing an algorithm

to define the algorithm in smaller words we use psuedocode

Pseudocode Convention

Comment - //

Blocks - { }

Identifiers - A, max(), node = record { val,
node *next }

for compound variable we use record

Assignment :- " := " is assignment operator

Logical operator :- =, <, >, ≤, ≥, ≠

Multidimentional arrays A[i,j,k]

indexes (Accessing elements of i,j,k)

code representation is A[i][j][k]

Loops - for Loop →

i=0 i<10 i++

For val := value1 to value2 step step do

{ }

while Loop →

while (condition) do

{ }

Do while Loop :-

Repeat { . } until (condition)

Conditional Statements -

IF → it condition then { . }

else it condition then { }

else { .. }

Switch → case {

condition 1 { . }

condition 2 : { . }

default ← else { }

}

Procedure / function :-

Algorithm functionName (parameters)

{ ..

return ;

}

* Given a set of numbers A and a number x find it the number x exists in set A

given , A = { $a_0, a_1, a_2, a_3, a_4, a_5$ }

```
Algorithm find (A, x) {
    length := A length,
    For i := 0 to length step 1 do {
        if A[i] = x then
            return true,
    }
    return false,
}
```

for validation find time complexity and memory complexity

state it sorted use BST

08/05/25

Recursive Algorithm

The algorithm that call itself directly or indirectly

Direct recursion :- $f(n) = f(n-1) + f(n-2)$

Indirect recursion :-

```
fun1() { fun2(), }
fun2() { fun1() }
```

any problem which can be solved by a loop can be done recursively

Approach to recursion :-

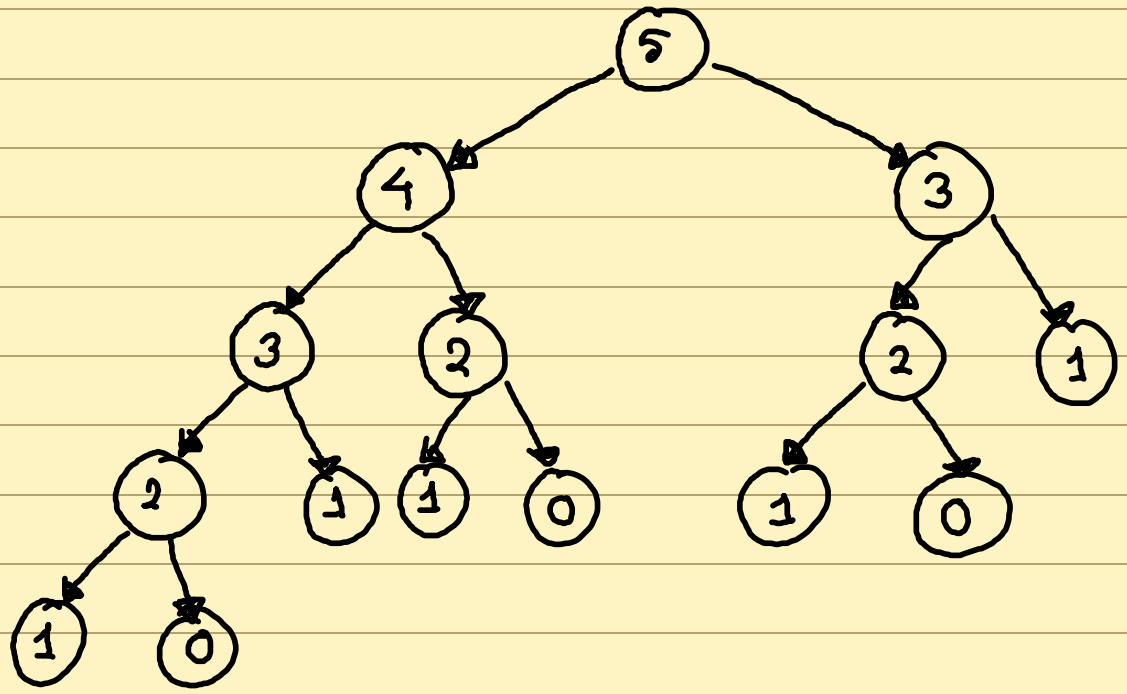
Fibonacci Sequence :- 0, 1, 1, 2, 3, 5, 8, 13, . , n^{th}

$n_i = n_{i-1} + n_{i-2}$

```
Algorithm nthFib(n) {
        if (n=0) { return 0, }
        if (n=1) { return 1, }

        nthf := nthFib(n-1) + nthFib(n-2),

        return nthf,
}
```



Example :- Binomial coefficient calculation :-

$$^{n}C_{r} = {}^{n-1}C_{r-1} + {}^{n-1}C_{r} \qquad [a\ b\ c\ d\ e]$$

n = 5

r = 3

if (n=1) return n,

if (r=1) return n,

GCD calculation : 12 , 9

```
9 | 12 | 1
    9
  ---
  3 | 9 | 3
      9
    ---
      0
```

gcd

gcd (a, b)

↳

```
a   b
9 ) 12 ( 1
    9
  ---
  3 ) 9 ( 3
      9
    ---
      0
```

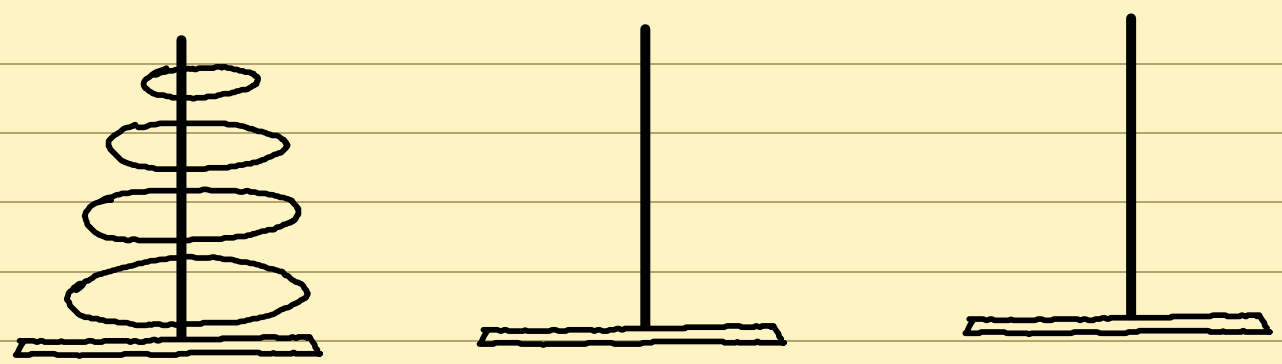
b

So,

```
Algorithm GCD (a, b) {
        if (a = 0) { return b, }
        return GCD ( b % a, a)
}
```

Tower of Hanoi :-

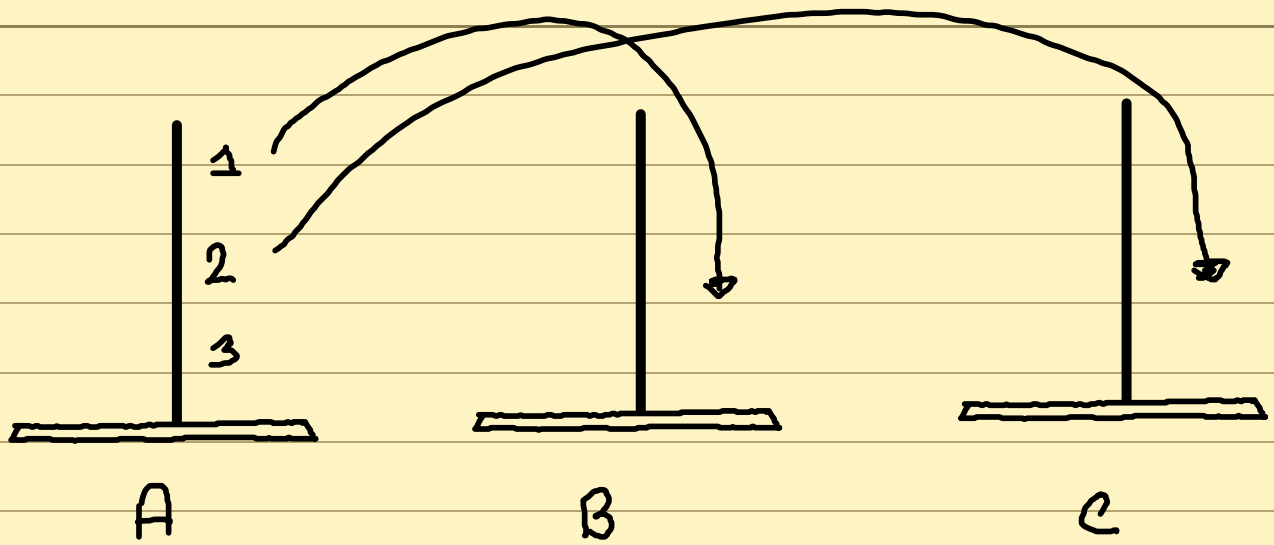
n - disk

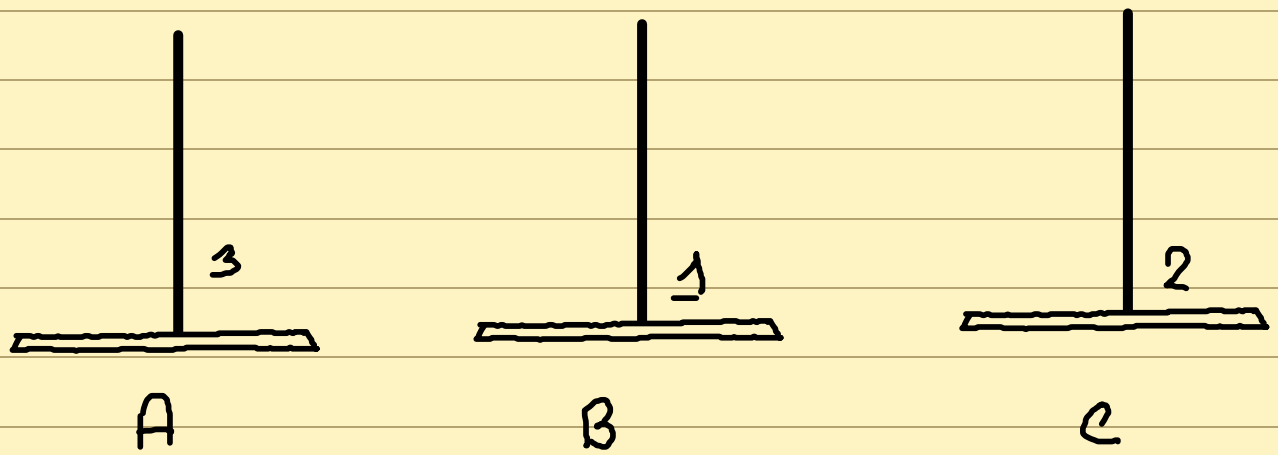


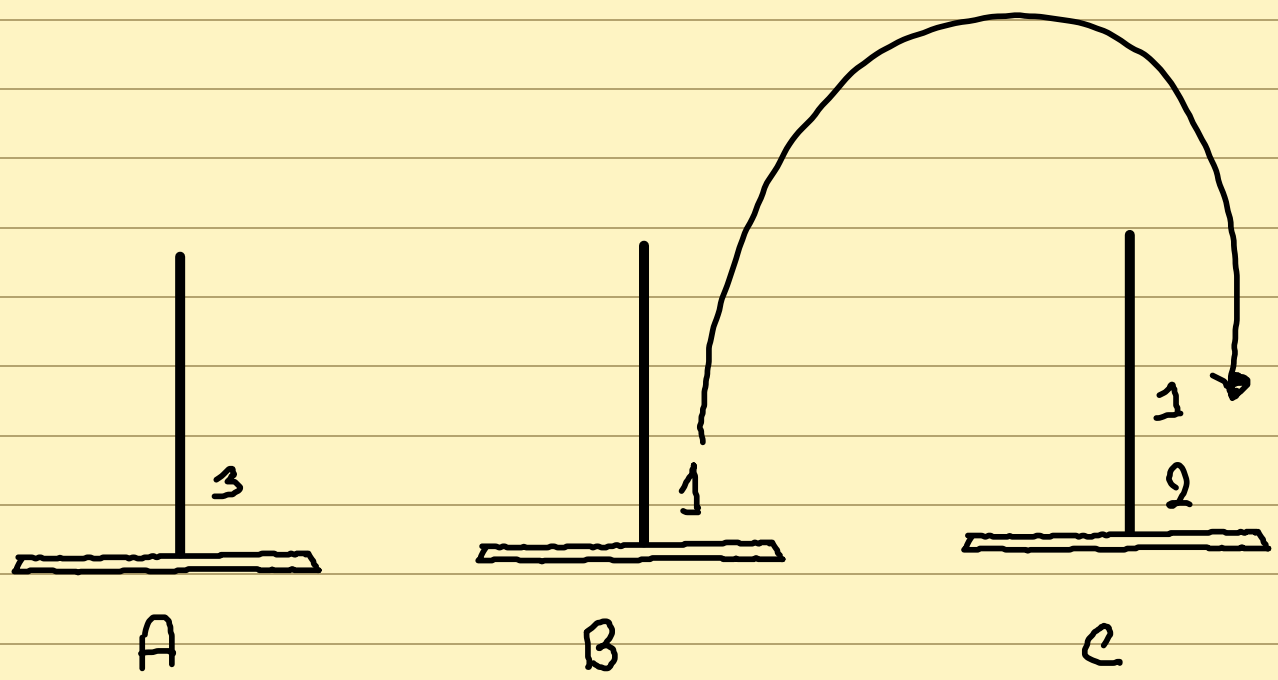
A 3 disk B C

```
Algorithm Tower (n, A, B, C) {
        if (n=0) { return , }

Tower (n-1, A, C, B)
→ write topmost disk from A to B,
Tower (n-1 , C, B, A )
```



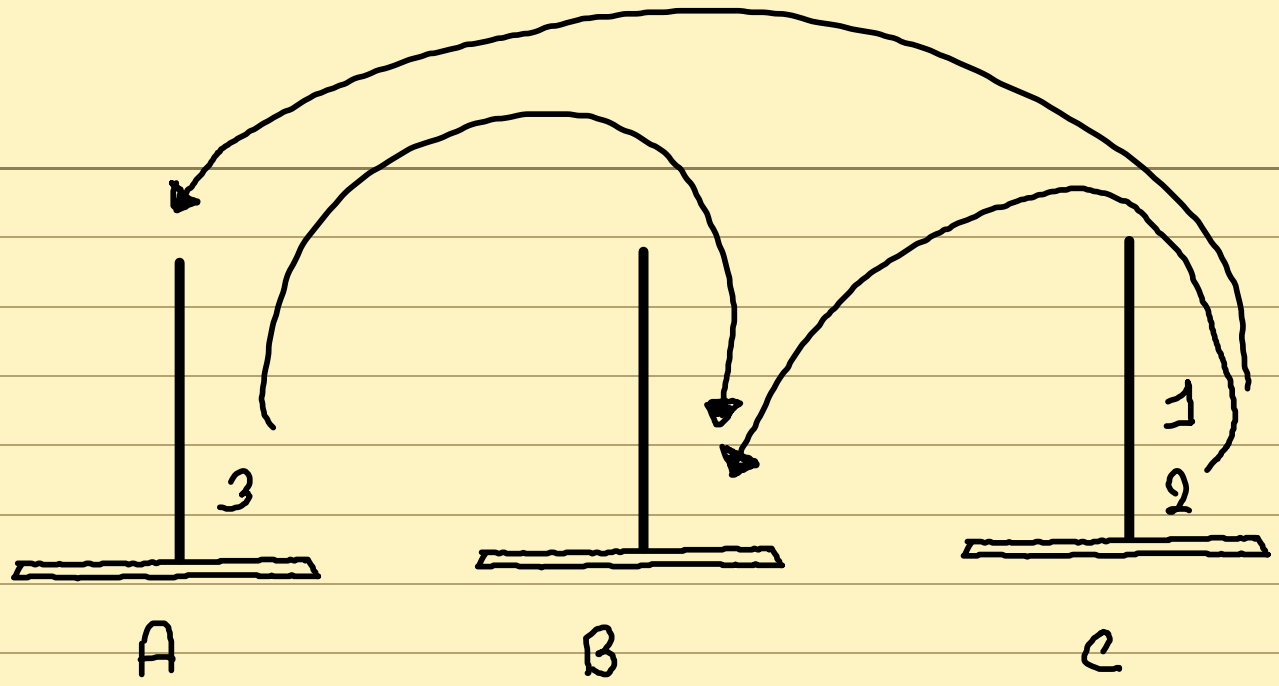


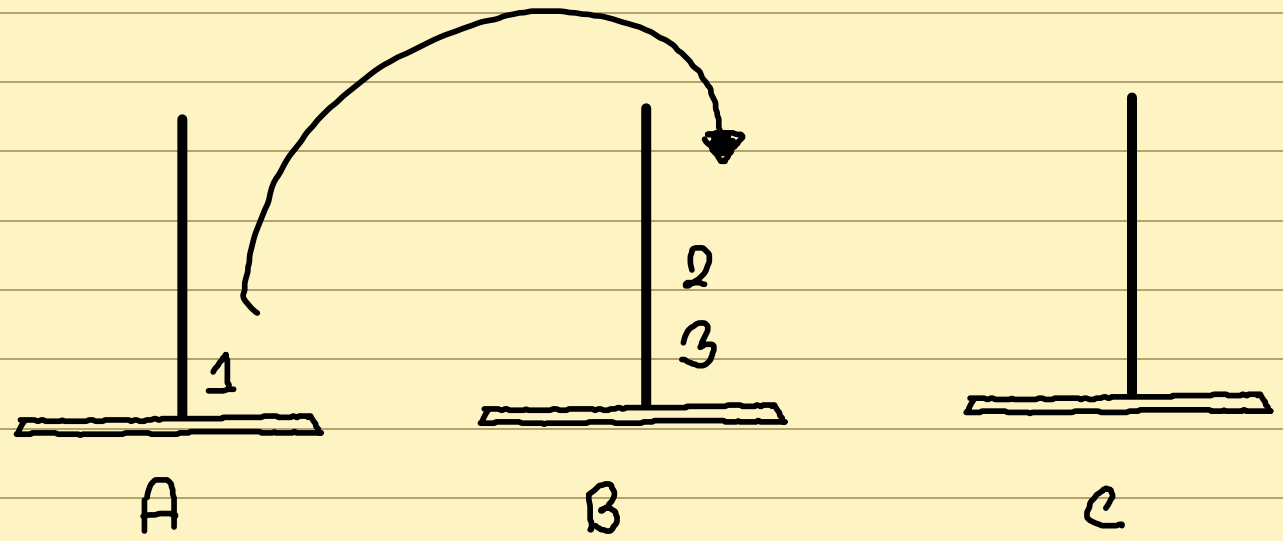


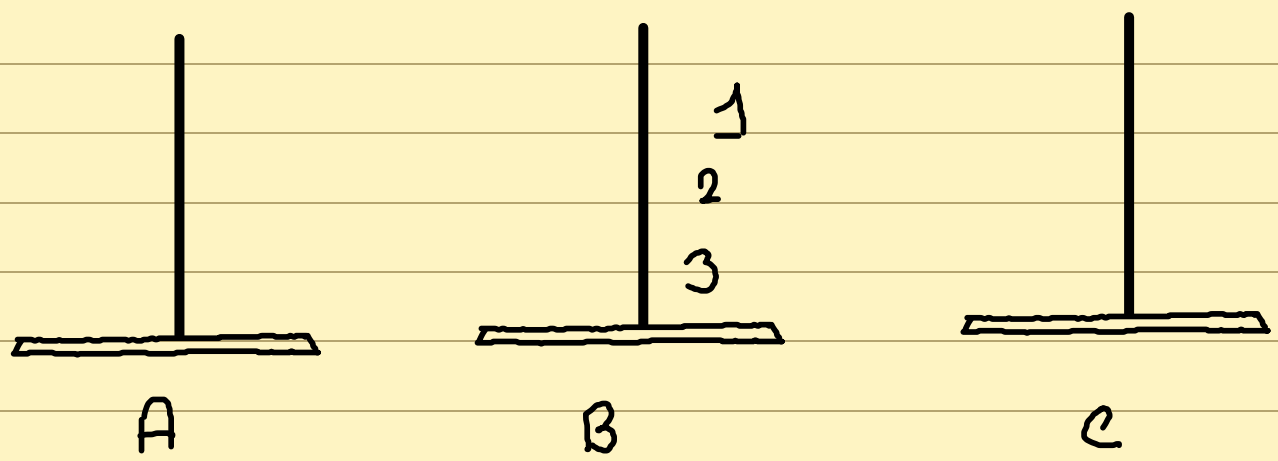
1 → A to B

2 → A to C

1 → B to C


3
1
2
A
B
C


1
2
3
A
B
C


1
2
3
A
B
C

Generating all possible permutation

a b c d e

if 3

a b c

Generating all possible ordering / permutation :-

A = {a, b, c, d, e, f}

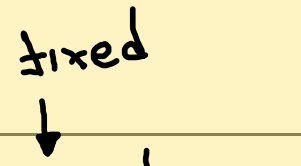
```
Algorithm permutation (A, k, n)
{   if (k = n-1) { write (A);
                   return ; }

for (i := 0 to n-1 do
        swap (A[k], A[i]),
        perm (A, k+1, n);
        swap (A[k], A[i]), )
```

if,
n = 6 = size
k = 3 = limit

initial call,

Perm (A, 0, n)



Lets see for,

a b c

b c

a b c

perm(A, 0, n)

→ perm(A, 1, n)

→ perm(A, 2, n)

Performance Analysis

Space complexity means, the amount of space/memory required from, starting and completion of execution of a program

Time complexity, required time for completion of execution of a program

1st step, determine estimated time

2nd step, write down the code and test the actual

```
Algorithm ABC (a,b,c) {
        return  a+b+ b*c + (a+b-c) / (a+b)+4 0;
                }
Algorithm Sum (A,n) {
        S = 0 0
   for  i:= 0  to  n-1  do {
                s:= s+ A[i];
   }
        return S.
}
Algorithm Rsum (A,n) {
        if(n<0) { return  0, }
        return  RSum (A,n-1) + A[n-1];
}
```

Space Complexity :-

Two major components;

- fixed memory component :- The memory that does not change prior to execution cases

In the algorithms above,

Sum → variable S,n and all the values inside for loop

abc → all the variable

- Variable memory component :- the memory that does change prior to execution cases

In the algorithms above,

Sum → A the array is variable memory,

RSum → A the array and function stack

Space complexity, $S(P) = C + S_P$

P → algorithm

C → fixed memory component

S_P → problem instance

Time Complexity :- $T(P) = C(P) + tp$ (problem instance)

↑ compilation time

but we do not consider compilation time as it is always constant

$tp = Ca * add(n) + Cs * Sub(n) + Cm * Mul(n) + Cd * Div(n)$

\+ -

but it is impossible to find runtime this way

So we consider finding by step by step time,

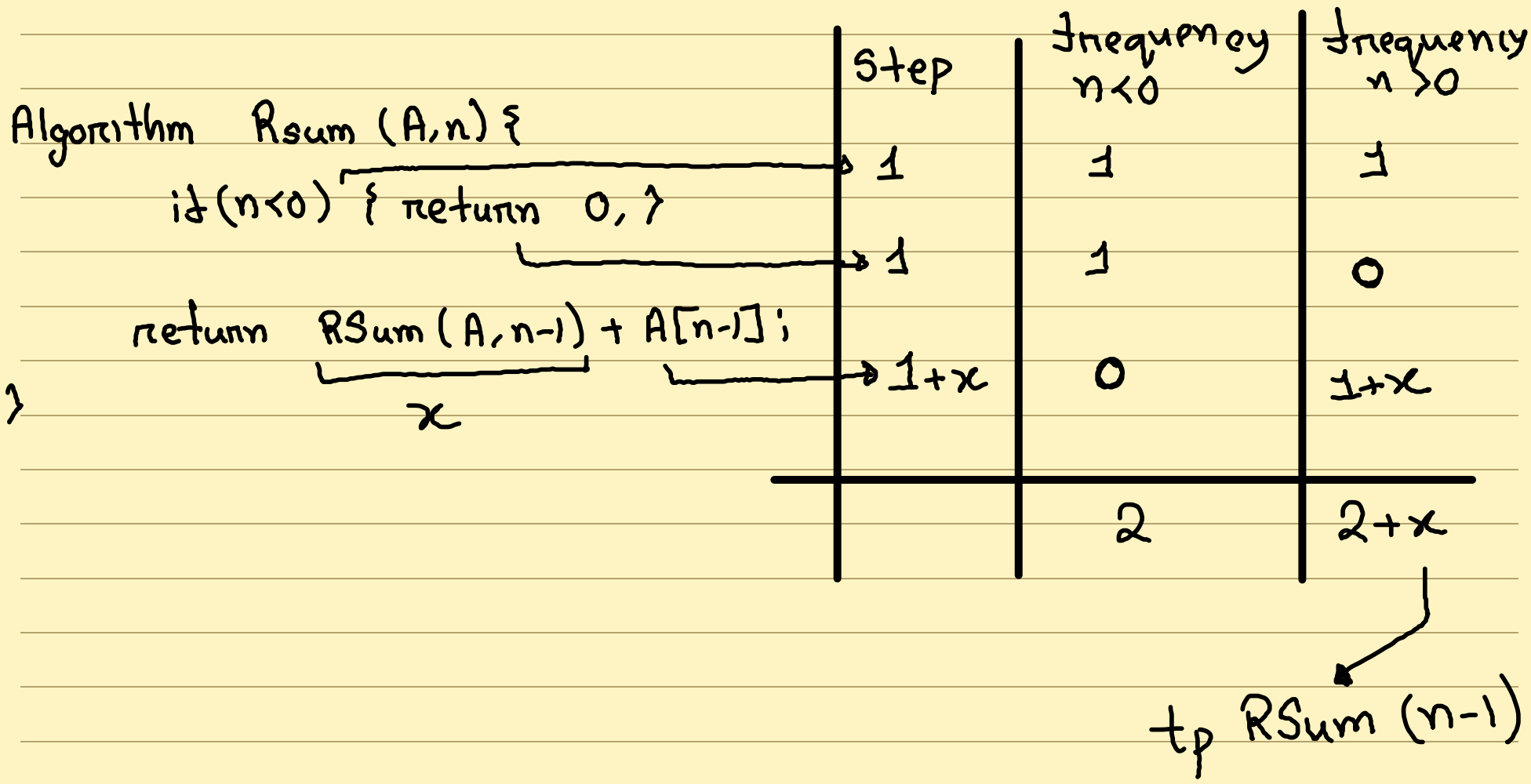
we consider comment as 0 step

```
Algorithm Sum (A, n) {                    0          -
    S = 0 0                               1          1
    for i := 0 to n-1 do {                1          n+1
        s := s + A[i];                    1          n
    }                                     0
    return S.                             1          1
}                                                    2n+3
```

step frequency

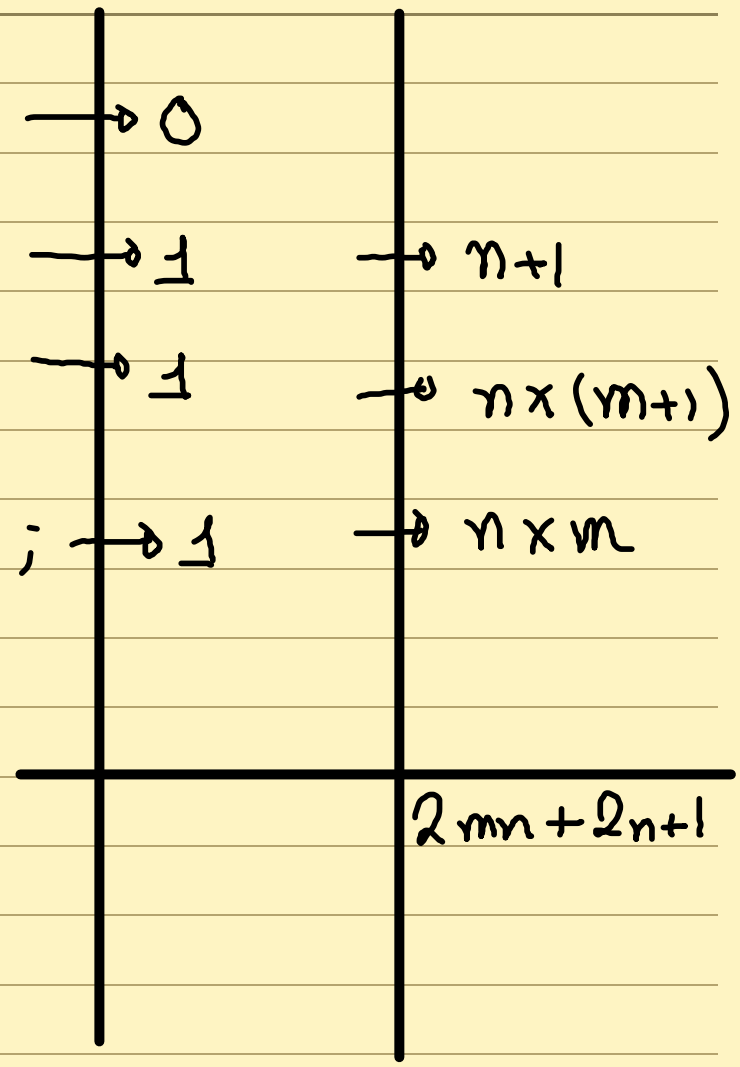
So total number of step 2n+3

So, time complexity O (2n+3) and we do not consider constants in big O function, So, time complexity will be O(n)



t_p RSum(n) → 2 + 2 + t_p RSum(n-1)

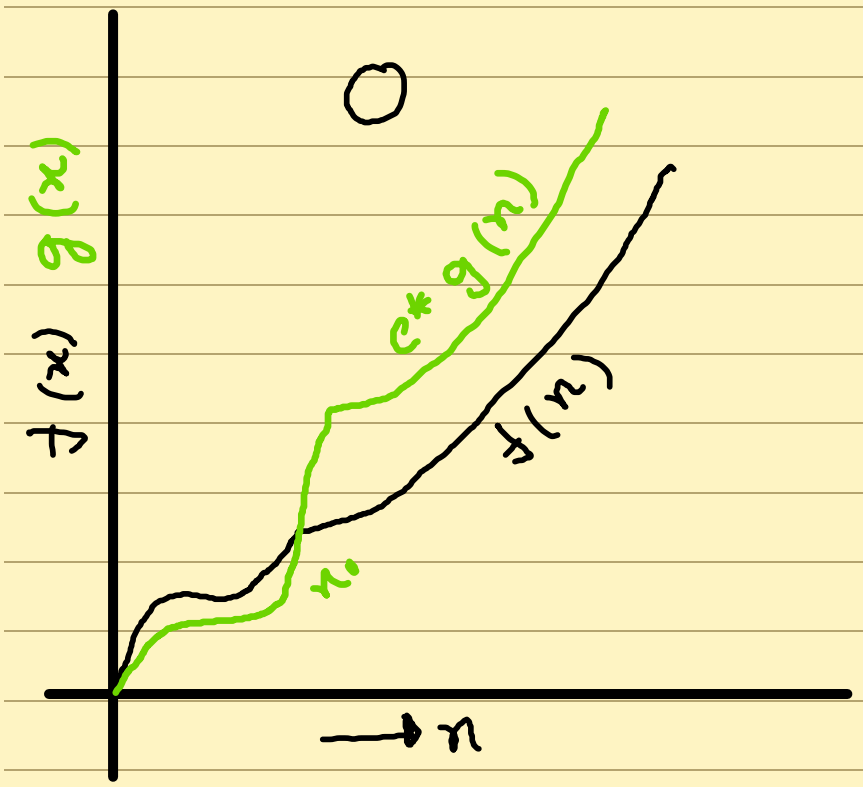
```
Algorithm (A, B, C, n, m) {
    for  i := 0  to  n  do {
        for  j := 0  to  m  do {
            C[i,j] = A[i,j] + B[i,j] ;
        }
}
```



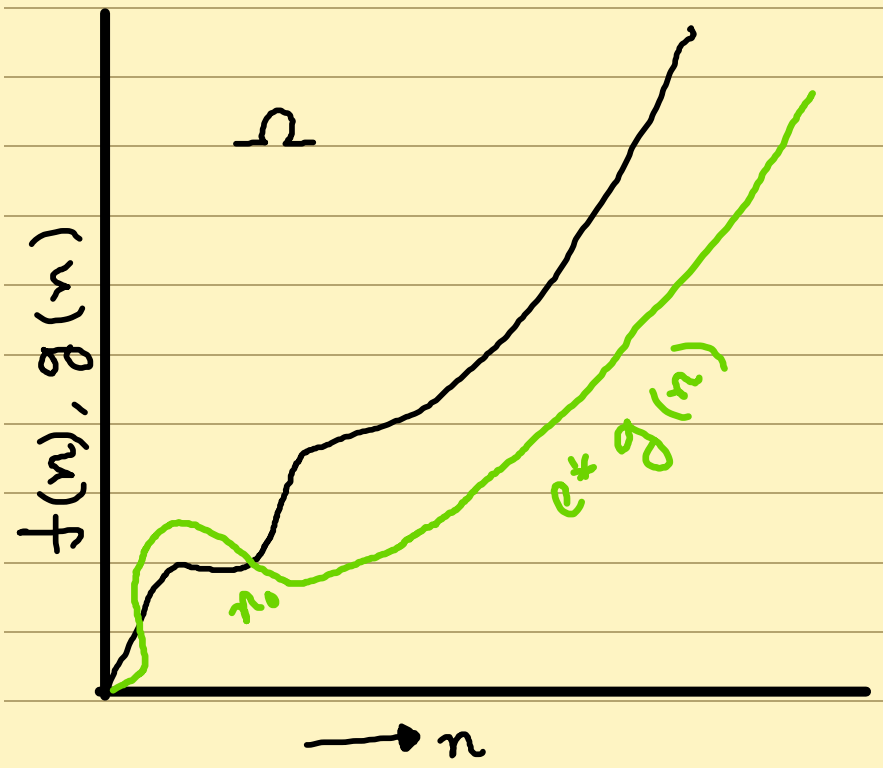
$O(m \times n)$

Asympotic Notation O, θ, Ω

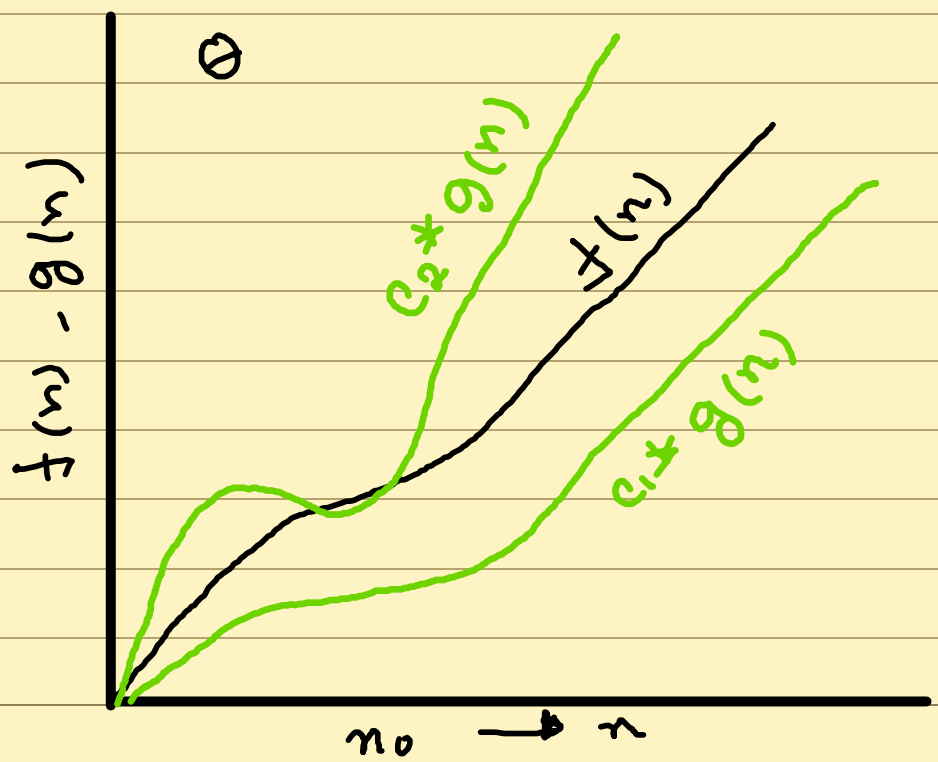
14/05/25



$$f(n) \leq c * g(n)$$



$$f(n) \geq c * g(n)$$



$$c_1 * g(n) \leq f(n) \leq c_2 * g(n)$$

$f(n) \longrightarrow O(g(n))$

Example : $f(n) = 2n+3$ $\quad$ $g(n) = 3n$

$n=1$, $f(n)=5$	$n=3$
$n=2$, $f(n)=7$	$n=6$
$n=3$, $f(n)=9$	$n=9$
$n=4$, $f(n)=11$	$n=12$

$f(n) \longrightarrow O(g(n))$

$O(3n)$

here if $g(n)=n^2$ it also maintains the condition

$f(n) \leq g(n) * c$

but we will have to take closest relation with $f(n)$

$\Omega \longrightarrow$

$f(n) = n^2+5$	$c_1 = 1$
$g(n) = n^2$	$c_2 = 2$
	$n_0 = 3$

$n^2+5 \leq c * n^2$

$\Rightarrow c * n^2 - n^2 \geq 5$

$\Rightarrow n^2 (c - 1) \geq 5$

$\Rightarrow c - 1 \geq 5/n^2$

$\Rightarrow c \geq \frac{5}{n^2} + 1$

So, $f(n) \geq 1 * g(n)$

$f(n) \leq 2 * g(n)$

$f(n) \rightarrow \Theta (g(n)$

$\Theta (n^2)$

big O → worst case

big Θ → average case

big Ω → best case

Divide and Conquer

falls under recursive algorithm

There are 3 steps of Divide and Conquer method,

- Divide the problem into smaller sub problem
- Solve the smaller Subproblem
- Combine the results of sub problems

```
Algorithm DandC (P)
{
   if small (P) then return S(P),
   else
        {
            Divide the problem P into subproblem P1, P2,
                P_K where K > 1
            call DandC over these P1, P2, P3, , Pk
            return Combine (DandC(P1), DandC(P2), ,
                                DandC (Pk)),
}
```

$$T(n) = \begin{cases} g(n), & n \text{ small enough} \\ T(n_1) + T(n_2) + \quad + T(n_k) & \text{otherwise} \end{cases}$$

Binary search is basic divide and conquer algorithm a small difference is it discards half of the problem in every recursive

step

```
Algorithm BS (A, l, r)
{
   if small (l, r) then return S(l, r),
   else
        {
            mid := (l+r)/2
            if (check (A, mid)) then
                return BS (A, l, mid),
            else
                return BS (A, mid, r);
        }
}
```

$\theta(g(n))$

$$T(n) = \begin{cases} T(1) & n \text{ is small enough} \\ T(n/2) + f(n) & \end{cases}$$

$f(n)$ → divide part for this case

it is $\theta(1)$

$T(n) \rightarrow T(n/2)$

$T(n/2) \rightarrow T(n/4)$

$T(n/4) \rightarrow T(n/8)$

$T\left(\frac{n}{2^{i-1}}\right) \longrightarrow T(n/2^i)$

$\frac{n}{2^i} = 1$

$\Rightarrow n = 2^i$

$\Rightarrow i = \log_2 n$ → for i recursive calls

So,

Time complexity = $f(n) * \log_2 n + g(n)$

$= \theta(1) * \log_2 n + \theta(1)$

$= \log_2 n$

Algorithm FindMinMax (A, l, r, max, min)

```
{
  if (r-l = 1) then
    { if (A[l] < A[r]) then
        { max := A[r],
          min := A[l],
        }
      else
        { max := A[l],
```

Solve Part

```
        min := A[r],
    }
else {
    mid = (l + r)/2,
```

D&C

```
    FindMaxMin(A, l, mid, max1, min1)
    FindMaxMin(A, mid+1, r, max2, min2),
```

combine

```
    if (max1 ≥ max2) then max := max1,
    else max := max2
    if (min1 ≤ min2) then min := min1
    else min := min2,
```

$$T(n) = \begin{cases} T(1) \longrightarrow \Theta(g(n)) = \Theta(1) & n \text{ small enough} \\ 2T(n/2) + f(n) \longrightarrow \Theta(1) \end{cases}$$

for ith recursion

$$T(n) = 2^i T(n/2^i)$$

So, $i = \log_2 n$

Time Complexity $= f(n) * \log_2 n * \underbrace{2^{\log_2 n}}_{n} + g(n)$

$= n \log_2 n + 1$

$= n \log_2 n$ [removing constants]

for merge sort

$f(n) \rightarrow \theta(n)$

$g(n) \rightarrow \theta(n)$

So time complixity is $n \log_2 n$

20/05/25

Peak finding

Problem P:- Given an array of n numbers find any peak $A[i]$ is a peak if,

$A[i] \geq A[i-1]$ where $i > 1$ and

$A[i] \geq A[i+1]$ where $i < n$

Straight forward approach →

```
1D Peak (A,n){
        for i := 1 to n do {
          flagleft = false, flagright = false
          if i>1 and A[i] ≥ A[i-1] then
                flagleft := true,
          if i<n and A[i] ≥ A[i+1] then
                flagright := true
          if flagleft and flagright then
                return A[i]
        }
          if n=1 or A[1] ≥ A[2] then
                return A[1]
```

if A[n] ≥ A[n-1] then

return A[n].

}

* n is the last index

time complexity O(n)

```
Algorithm OneDPeak (A, l, r) {
        if (l=r) then
                return A[l].
        m := (l+r)/2 ,
        if m>1 and m<n and A[m] ≥ A[m-1] and
          A[m] ≥ A[m+1] then
                return A[m]
        else if m>1 and A[m] ≤ A[m-1] then
                return OneD Peak (A, l, m-1),
        else .
                return OneD Peak (A, m+1, r)
        }
```

Time complexity, $T(n) = \begin{cases} T(1) & A[m] \text{ is a peak} \\ T(n/2) + \theta(1) & A[m] \text{ is not a peak} \end{cases}$

for ith recursive call

$$T = T(n/2^i)$$

$$1 = n/2^i$$

$$\Rightarrow i = \log_2 n$$

Finding 2D Peak →

Problem P1 → Given a 2D array of size m, n find a peak [i, j] is peak if

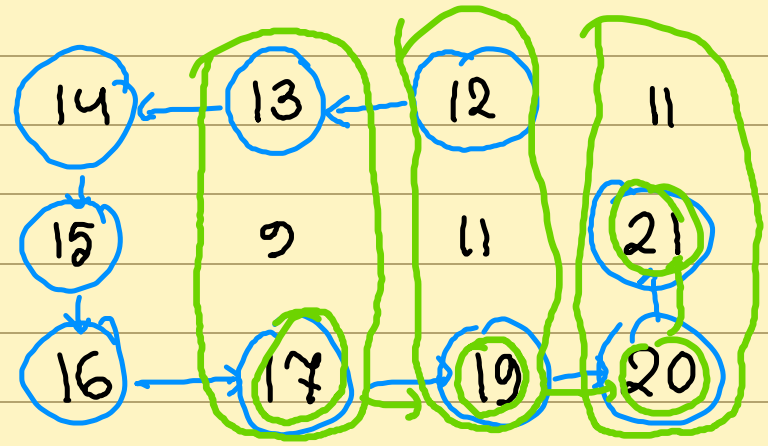
$A[i,j] \geq A[i-1,j]$ when $i > 1$

$A[i,j] \geq A[i+1,j]$ when $i < m$

$A[i,j] \geq A[i,j-1]$ when $j > 1$

$A[i,j] \geq A[i,j+1]$ when $j < n$

example - array → 10 8 10 10



randomly pick an element check it it a peak, it not move to the largest neighbour and repeat the process

max value

To make this algorithm faster,

we will find the global peak of the mid column check its neighbours take the largest neighbours column to repeat the process it global peak of column is a peak then return it

$$T(n,m) = \begin{cases} \Theta(n) \\ T(n, m/2) + f(n) \end{cases}$$

$\Theta(n)$

$T(n, m/2) + \Theta(n) \longrightarrow 1$

$T(n, m/2) + 1\Theta(n)$

$i = \log_2 m$ $\qquad 1\Theta(n)$

$= i \times \Theta(n)$

$= n \log_2 m$

if row > col → divide based on row

row < col → divide base on col

Greedy Algorithm

Feasible Solution → Subset of the problem that contained conditions fixed by the problem

Objective function → Tries to maximize or minimize a functions value base on the feasible solution

Optimal Solution → Among all the feasible solutions those solutions that minimize and maximizes the objective function

Subset Paradigm

Ordering Paradigm

Functional Knapsack

n objects, A knapsack bag with capacity m

weight of n objects $w = \{w_1, w_2, w_3, \ldots, w_n\}$

Profit of n objects $n = \{P_1, P_2, P_3, \ldots, P_n\}$

Objective maximize $\sum_{i \in \text{objectives}} x_i * P_i$

feasibility $\longrightarrow \sum_{i \in \text{objectives}} x_i * w_i \leq m$

$n = 3 \quad m = 20 \quad P = \{25, 24, 15\}$

$w = \{18, 15, 10\}$

feasible solution →

$w = 9, P = 12.5 \quad w = 5, P = 8 \quad w = 2.5, P = 3.75$

		total weight	total Profit
→ (1/2, 1/3, 1/4)	→	16.5	24.25
→ (1, 2/15, 0)	→	20	28.2
→ (0, 2/3, 1)	→	20	31
→ (0, 1, 1/2)	→	20	31.5

↳ this has the maximum profit

lets calcutate the maximum profit per kg

$\frac{P_1}{w_1} = \frac{25}{18} = 1.38$ $\quad \frac{P_3}{w_3} = \frac{15}{10} = 1.5$

$\frac{P_2}{w_2} = \frac{24}{15} = 1.6$

Subset paradigm → Instead of taking the whole problem set, we will take a subset such that it maximizes objective function

So, we will take whole of maximum then remain of maximum

```
Algorithm Greedy (n, A) {  solution := ∅
                for (i  in A ) {
                    if (select (i)) {
                        solution := Union(solution,i),}
                    return Solution
                }
```

n task/jobs having individual deadline, and each job taking 1 unit time to finish Now given profit P for each job task you need to minimize the total profit

Feasibility No two jobs can run parallely

Objective maximize total profit

$n = 4$ $P = \{100, 10, 15, 27\}$ $d = \{2, 1, 2, 1\}$

Feasible Solution

task number	Sequence	Profit
{1, 2}	2, 1	110
{1, 3}	1, 3 or 3, 1	115
{1, 4}	4, 1	127
{2, 3}	2, 3	25
{3, 4}	4, 3	42

P = {100, 10, 15, 17} d = {2, 1, 2, 1}

n jobs with deadline for each d_i and profit p_i. Each jobs need 1 unit time to finish

what is the maximum profit which can be gained without overlapping jobs

Ans →

Job	seq	Profit
{1, 2}	2, 1	110
{1, 4}	4, 1	127
{1, 3}	1, 3 or 3, 1	115

optimal {1, 4} with sequence {4, 1} profit 127

```
Algorithm JobSchedule (J, d, P, n) {
    sort J descending based on profit,
    for i := 1 to n do
        for s := d[i] to 1 do
            if slot [s] is empty then
                assign slot[s] to job J[i]
                profit := profit + P[i],
```

```
            break;
        return profit;
    }
```

Theorem :- J is a set of k jobs and $\sigma = i_1, i_2, i_3, \ldots i_k$ is a permutation of 1 to k such that $d_{i_1} \leq d_{i_2} \leq \ldots \leq d_k$ Now J is a feasible solution iff jobs in J can be processed with the sequence σ without any overlaps

$d = \{7, 8, 9, 12, 11, 10, 6\}$

jobs = 1 2 3 4 5 6 7

$6 < 7 < 8 < 9 < 10 < 11 < 12$

$\sigma = \{7 \quad 1 \quad 2 \quad 3 \quad 6 \quad 5 \quad 4$

$q = \{1, 2, 3, 4, 5, 6, 7\}$ ⟶ q is time slot number

Proof ⟶ J is feasible for sequence $\sigma' = r_1, r_2, \ldots, r_k$ if

$d_{r_q} \geq q$ where $1 \leq q \leq k$

Assume that,

$\sigma' \neq \sigma$

The least index a where $i_a \neq r_a$, so there exists, $r_b = i_a$ and $b > a$

$\sigma = i_1, i_2, i_3, \ldots, i_K$

$\sigma' = r_1, r_2, r_3, \ldots, r_K$

so, $a = 3$

So we can also say,

$dr_b \geq dr_a$

this is not always true

$dr_b \geq b \rightarrow b > a$

$dr_a \geq a \rightarrow dr_b \geq dr_a$

Let's track back as I don't understand a shit sirc is saying

Given, $\sigma' \neq \sigma$ we have a where $r_a \neq i_a$

so, there exists $r_b = i_a$ and exist $r_a = i_c$

by swapping them we can make σ' equal to σ